

Reviewer Pack — Minimal Rank of Tropicalized K-theory Invariants vs BP_ReadT...

Entry 7ffdd386896b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 18:43:21 UTC

Minimal Rank of Tropicalized K-theory Invariants vs BP_ReadTwice Complexity

Entry ID: 7ffdd386896b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 18:43:21 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (K-theory) **Field B** (complexity object): Complexity Theory: BP_ReadTwice Complexity

Statement:

[‘For a read-twice branching program P , the minimal rank of its tropicalized K-theoretic invariant $\rho(P)$ is $O(\log \text{size}(P))$. Specifically, $\rho(\text{IP_2 trivial BP}) = \Omega(n)$.’, ‘where IP_2 trivial BP refers to an inner product over $\{0,1\}^n$ with trivially small read-twice complexity.’, ‘The minimal rank of tropicalized K-theoretic invariants is defined as the smallest dimension of a vector space spanned by all homomorphisms from P to the tropical polynomial ring.’]

Rationale (proposer’s reasoning):

[‘Tropical K-theory has been used to analyze complexity classes, particularly for proving lower bounds on circuit complexity. By leveraging its algebraic structure, we may expose hidden complexity in branching programs.’, ‘Previous work has shown that certain geometric invariants can be related to circuit complexity. The conjecture aims to extend this connection by considering tropicalized K-theory invariants for BP_ReadTwice Complexity.’, ‘If true, the conjecture would provide a novel approach to understanding the complexity of read-twice branching programs and potentially lead to new lower bounds.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e52497430cc143c8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of $\rho(P)$ over 30 seeds, normalized by $\log(\text{size}(P))$, is within $[0.5, 1.5]$, and it is falsified if any seed results in a ratio exceeding 2 or less than 0.25.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropical geometry' AND 'K-theory' AND 'BP_ReadTwice complexity' AND 'minimal rank' - 'read-twice branching program' AND 'tropicalized K-theory invariants' AND 'complexity theory' AND 'polynomial ring' - 'inner product over $\{0,1\}^n$ ' AND 'trivial read-twice complexity' AND 'tropical K-theory' AND 'minimal rank'

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1910.01251v2>] Search problems in algebraic complexity, GCT, and hardness of generator for invariant rings - [<http://arxiv.org/abs/1308.2123v1>] The Control of Branching Morphogenesis - [<http://arxiv.org/abs/1402.3280v3>] K -theory and homotopies of 2-cocycles on transformation groups - [<http://arxiv.org/abs/2012.11098v2>] Sublinear Maximum Inner Product Search using Concomitants of Extreme Order Statistics - [<http://arxiv.org/abs/2410.18987v5>] Point Cloud Synthesis Using Inner Product Transforms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda x: abs(A[x][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            for j in range(n):
                A[i][j] *= factor
            for k in range(m):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A
```

```

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if m == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def random_read_twice_bp(n):
    bp = []
    for _ in range(2):
        bp.append([random.choice([0, 1]) for _ in range(n)])
    return bp

def tropicalized_k_theoretic_invariant(bp):
    n = len(bp[0])
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(n):
            if bp[0][i] == bp[1][j]:
                A[i][j] = 1
    A = gaussian_elimination(A)
    rank = sum(1 for row in A if any(row))
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    bp = random_read_twice_bp(n)
    rho = tropicalized_k_theoretic_invariant(bp)
    results.append({"n": n, "rho": rho})

mean_rho = sum(result["rho"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = 0.25 <= mean_rho / math.log(instances_tested) <= 1.5
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "rho_over_log_n",
    "metric_value": mean_rho,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
}

```

```

        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

    mean_rho = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rho} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b80d913.py", line 100, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b80d913.py", line 78, in run_trial
    rho = tropicalized_k_theoretic_invariant(bp)
          ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b80d913.py", line 70, in tropicalized_k_theoretic_invariant
    A = gaussian_elimination(A)
        ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b80d913.py", line 26, in gaussian_elimination
    factor = 1 / A[i][i]
              ~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the conjecture could neither be supported nor falsified based on the available information. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11512
2	preregistration	ollama_remote	glm4:latest	0	0	5691
3	novelty	ollama_remote	glm4:latest	0	0	5210
4	novelty	ollama_remote	glm4:latest	0	0	6601
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15566
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12963
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9073
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12500
9	judge	ollama_remote	glm4:latest	0	0	25123

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104238 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7ffdd386896b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7ffdd386896b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7ffdd386896b.tar.gz` (if generated)